

Understanding and fitting the New York Fed MCT inflation model

From regression to latent factors, state space, and Bayesian sampling

Written for a reader with substantial statistical experience who is new to factor models.

The central problem is to infer a persistent inflation signal from many noisy sectoral inflation series. The signal is not observed, its relationship with each sector can change, and the noise itself changes over time. This guide explains how the model expresses that problem and how its estimation algorithm solves it in blocks.

Key idea. A factor is an unobserved regressor shared by several observed series. A loading is the coefficient linking a series to that factor. Neither is the expenditure weight used to aggregate inflation.

Scope. The mathematical target is the monthly, 17-sector configuration selected by the NY Fed's publicly released PCE replication code: three moving-average lags, no time aggregation, and no extra cross-dependent sector pairs. Sources are linked at the end. This is an explanatory document prepared for this project, not a NY Fed publication.

Status distinction. The Python implementation has passed selected matrix and Gaussian-smoother comparisons against the published code. These checks do not establish convergence of the full Bayesian fit or an exact replication of published MCT estimates. The newer-data reconstruction also has a documented housing-input approximation.

Contents

1	A factor model is a regression with missing regressors	2
2	Factor, loading, weight: three different objects	3
3	How PCA, factor models, and state space fit together	4
4	The actual MCT model: what generates sector inflation?	5
5	Changing volatility and outliers are different mechanisms	6
6	Data preparation: what goes into the estimator?	7
7	What is Bayesian about it?	8
8	One sampling sweep, step by step	9
9	What the Kalman filter and simulation smoother actually do	10
10	The other Bayesian updates, in equations	11
11	Turn posterior draws into the reported inflation measure	12
12	Convergence, validation, and computational cost	13
A	The state vector and an implementation map	14
B	Reading route and sources	15

1 A factor model is a regression with missing regressors

Suppose you observe inflation in three sectors. If an economy-wide inflation pressure f_t were observed, you could write three familiar regressions:

$$y_{1t} = \mu_1 + \lambda_1 f_t + e_{1t}, \quad y_{2t} = \mu_2 + \lambda_2 f_t + e_{2t}, \quad y_{3t} = \mu_3 + \lambda_3 f_t + e_{3t}.$$

The factor-model step is to admit that f_t is *not* observed. You jointly infer its values and the coefficients λ_i from the co-movement of the observed series. In vector form, a static Gaussian model is

$$\mathbf{y}_t = \boldsymbol{\mu} + \boldsymbol{\lambda} f_t + \mathbf{e}_t, \quad f_t \sim \mathcal{N}(0, 1), \quad \mathbf{e}_t \sim \mathcal{N}(0, \Psi), \quad (1)$$

where Ψ is diagonal in the basic version and factor and errors are independent. Consequently,

$$\text{Cov}(\mathbf{y}_t) = \boldsymbol{\lambda} \boldsymbol{\lambda}' + \Psi.$$

The shared factor explains off-diagonal covariance; sector-specific errors explain additional variation within individual series. With k factors, replace $\boldsymbol{\lambda} f_t$ by $\Lambda \mathbf{f}_t$, where Λ is $n \times k$.

What does “extracting a factor” mean mathematically? Conditional on known loadings and Ψ , the posterior for the single factor in (1) is

$$f_t \mid \mathbf{y}_t \sim \mathcal{N}(m_t, V), \quad V = (1 + \boldsymbol{\lambda}' \Psi^{-1} \boldsymbol{\lambda})^{-1}, \quad m_t = V \boldsymbol{\lambda}' \Psi^{-1} (\mathbf{y}_t - \boldsymbol{\mu}).$$

This follows by completing the square in the Gaussian likelihood times the $\mathcal{N}(0, 1)$ prior. It is a precision-weighted combination of the sector readings. For example, if $\boldsymbol{\lambda} = (1, 1)'$, $\Psi = \text{diag}(1, 4)$ and demeaned readings are $(2, 4)'$, then $V = 1/2.25$ and $m_t = 3/2.25 = 1.33$. The noisier second reading receives less influence, and the prior shrinks the result toward zero.

Conversely, if the factor path were known, estimating each loading would look like regression. Alternating these conditional problems is one route to joint estimation. Dynamic models add information from neighbouring dates.

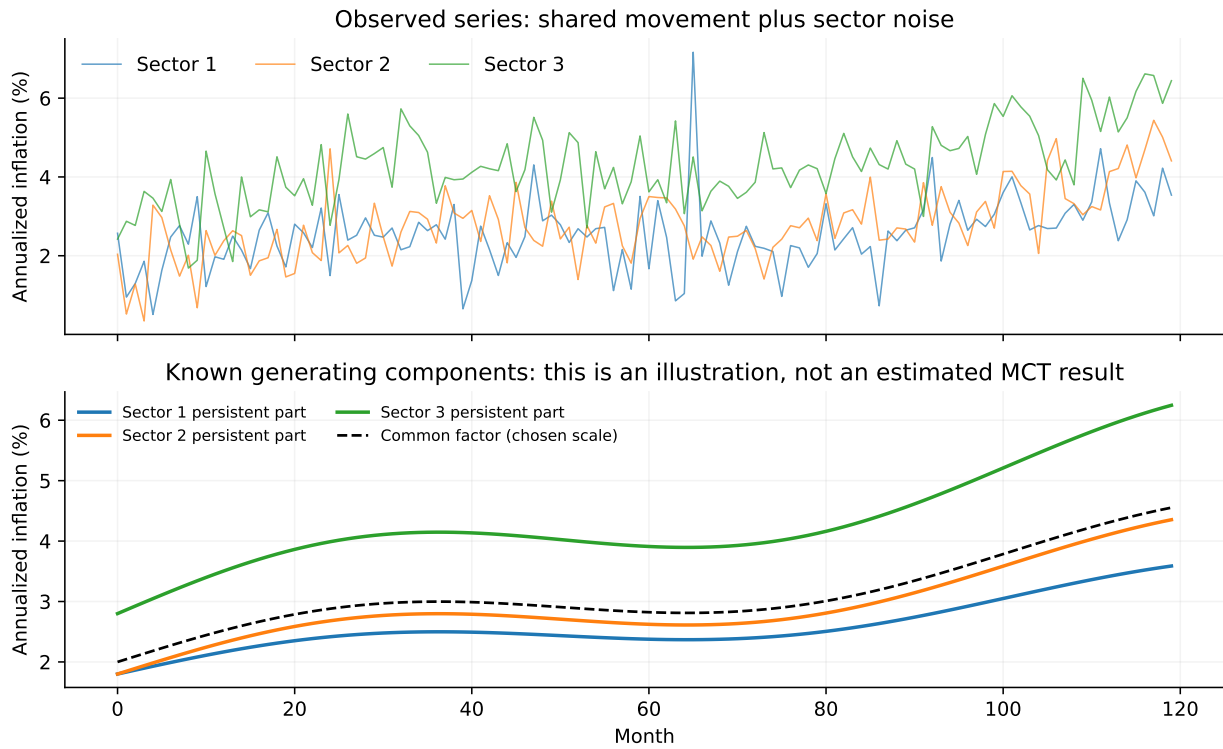
A factor is not automatically an economic cause. Shared movements can support a common statistical factor without uniquely identifying demand, monetary policy, or supply shocks. Such causal labels require additional identification assumptions.

Key idea. The factor is learned from the data’s shared variation. It is not an externally observed index that gets inserted into an ordinary regression.

2 Factor, loading, weight: three different objects

Object	Mathematical role	Interpretation
Factor f_t	Shared unobserved regressor	A common pattern across sectors.
Loading λ_i	Coefficient in $y_{it} = \lambda_i f_t + \dots$	The sector's sensitivity to that pattern.
Weight w_{it}	Coefficient in a final aggregation	The sector's share of core expenditure.

Loadings do not need to sum to one and can have either sign. Core expenditure weights do sum to one. A large sector need not have a high loading, and a small sector may provide valuable information about the factor if it is informative and relatively quiet.



The figure is generated from invented data with known components. The first sector has a large isolated spike, but all three share a gradual rise. A fitted model must infer these components; it does not get to observe the smooth lines shown in the lower panel.

Identification matters. The products are unchanged if you multiply f_t by c and divide every loading by c . The sign can also flip. Static models often fix $\text{Var}(f_t) = 1$ and choose a sign convention. With several factors, rotations introduce additional ambiguity. For MCT, interpret sector products and the aggregate rather than treating the raw common factor's level as the national inflation rate. The published code also warns that the allocation of levels between common and sector-specific trends is not identified without normalizations.

3 How PCA, factor models, and state space fit together

PCA finds linear combinations that explain maximal observed variance. A probabilistic factor model specifies how latent variables generate observed data, including a model for residual variance. Their solutions coincide only under particular assumptions. In particular, the first principal component need not be persistent inflation: a large common temporary shock can dominate observed variance.

To introduce time, let a factor evolve according to an equation such as

$$f_t = \rho f_{t-1} + \eta_t, \quad \eta_t \sim \mathcal{N}(0, q).$$

Now nearby observations help identify the factor. This is a *dynamic* factor model. It is also a state-space model:

$$\begin{aligned} \mathbf{y}_t &= \mathbf{Z}\mathbf{x}_t + \boldsymbol{\epsilon}_t && \text{observation equation,} \\ \mathbf{x}_{t+1} &= \mathbf{F}\mathbf{x}_t + \mathbf{G}\boldsymbol{\eta}_t && \text{transition equation.} \end{aligned}$$

The state $\mathbf{x}_t$ contains the factor and any other hidden quantities needed to explain the data.

Term	What it tells you
Factor model	Some latent influences are shared across observed series.
State-space model	Hidden states evolve through time and generate observations.
Bayesian estimation	Unknown states and parameters receive a posterior distribution.
Kalman filter	Computes conditional Gaussian updates given the state-space matrices.
MCMC	Samples the joint posterior when the whole model cannot be solved in one Gaussian calculation.

These terms describe different aspects of the same model. A state-space model need not contain a common factor. A factor model need not be dynamic or Bayesian. The MCT model is all three.

Connection with VARs. A VAR models observed variables using their observed lags. A dynamic factor model can instead put a VAR on a small set of hidden factors and connect them to many observed series. MCT uses random-walk trend states and finite-memory transitory components rather than simply fitting an unrestricted VAR to the 17 observed inflation series.

Key idea. MCT estimates persistence explicitly. Maximising explained cross-sectional variance alone would not distinguish persistent pressure from temporary co-movement.

4 The actual MCT model: what generates sector inflation?

Let $i = 1, \dots, 17$ index sectors and t index months. A compact representation of the published monthly configuration is

$$y_{it} = a_{it}\tau_{ct} + \tau_{it} + b_{it}e_{ct} + e_{it} + \sum_{\ell=1}^3 \theta_{i\ell}e_{i,t-\ell} + \delta_{it}. \quad (2)$$

Here $\delta_{it} \sim \mathcal{N}(0, 10^{-6})$ is the small observation variance used in the code. The substantive decomposition is:

Component	Meaning
$a_{it}\tau_{ct}$	Common persistent influence, scaled for sector i .
τ_{it}	Additional persistent movement specific to sector i .
$b_{it}e_{ct}$	Common transitory influence, scaled for sector i .
$e_{it} + \sum_{\ell=1}^3 \theta_{i\ell}e_{i,t-\ell}$	Sector-specific transitory component with MA(3) dynamics.

The two trends and the two loading paths evolve as

$$\tau_{ct} = \tau_{c,t-1} + \sigma_{\Delta\tau,c,t}z_{\Delta\tau,c,t}, \quad (3)$$

$$\tau_{it} = \tau_{i,t-1} + \sigma_{\Delta\tau,i,t}z_{\Delta\tau,i,t}, \quad (4)$$

$$a_{it} = a_{i,t-1} + \lambda_{a,i}z_{a,i,t}, \quad b_{it} = b_{i,t-1} + \lambda_{b,i}z_{b,i,t}. \quad (5)$$

Innovations z are standard normal and independent across their basic innovation channels. Dependence among sectors arises through the shared components. The λ parameters govern the smoothness of the loadings; they are estimated rather than re-estimated independently at every date.

Why call the first two terms persistent? An innovation to a random-walk trend remains in its level unless subsequent innovations offset it. In contrast, a sector transitory innovation enters the observation for at most four months through the MA(3) term. “Transitory” therefore does not mean “lasts only one month.”

The sector’s persistent inflation is

$$g_{it} = a_{it}\tau_{ct} + \tau_{it}.$$

Removing sector-specific trends would change the target: MCT is not just the common factor. The model estimates two shared channels—persistent and transitory—alongside sector-specific channels. Sources: [1, 2].

5 Changing volatility and outliers are different mechanisms

For the common and sector-specific transitory innovations, the code uses

$$e_{jt} = r_{jt}\sigma_{e,j,t}z_{e,j,t}, \quad j \in \{c, 1, \dots, 17\}.$$

The ordinary shock scale is $\sigma_{e,j,t}$. The multiplier r_{jt} allows an occasional outlier. Its support is $\{1\} \cup \{39 \text{ equally spaced values from } 2 \text{ to } 10\}$. If p_j denotes the probability of an ordinary observation,

$$\Pr(r_{jt} = 1) = p_j, \quad \Pr(r_{jt} = r) = \frac{1 - p_j}{39} \quad (r > 1).$$

A large observation can then be explained partly by a transitory outlier without forcing a comparably large trend revision. It is not automatically labelled an outlier; that allocation is inferred jointly with everything else.

For each trend-innovation or transitory-innovation scale, log variance follows a random walk:

$$h_{jt} = \log \sigma_{jt}^2, \quad h_{jt} = h_{j,t-1} + \gamma_j u_{jt}, \quad u_{jt} \sim \mathcal{N}(0, 1).$$

There are $2(17 + 1) = 36$ such volatility paths. A period of sustained turbulence can raise the ordinary volatility path; an isolated extraordinary observation can instead receive a large r_{jt} . The prior and the surrounding data affect that distinction.

Why this complicates fitting. With known variances and loadings, the hidden inflation states are linear Gaussian. But $a_{it}\tau_{ct}$ multiplies two unknown quantities, and $\exp(h_{jt})$ enters a variance. The whole joint problem is not one ordinary Gaussian regression or one Kalman-filter pass.

A useful intuition. A jump in one noisy sector provides weaker evidence for common persistent inflation than a sustained, coherent movement across many relatively quiet sectors. Yet the model does not apply a hard threshold such as “three rising months equals trend.” It allocates probabilities across alternative decompositions under its specified dynamics.

Key idea. The trend is estimated under assumptions about persistence and noise. It is not an observable truth obtained by mechanically smoothing an aggregate.

6 Data preparation: what goes into the estimator?

For a seasonally adjusted sector price index P_{it} , the supplied replication data use annualized monthly inflation:

$$y_{it} = 100 \left[\left(\frac{P_{it}}{P_{i,t-12}} \right)^{12} - 1 \right]. \quad (6)$$

For a 0.2% monthly increase, this is $100(1.002^{12} - 1) \simeq 2.43\%$. It is not the twelve-month change $100(P_{it}/P_{i,t-12} - 1)$, although both are reported in annual percentage units. A sudden monthly observation can be large in annualized terms.

The original replication snapshot contains enough earlier observations to construct a sample from January 1960 through October 2023: $T = 766$ and $n = 17$. The observation matrix is therefore 766×17 . The newer local reconstruction runs through July 2026 and has 799 estimation months. Index reference bases cancel in price ratios, but rebasing must not introduce an artificial discontinuity within a series.

All sectors inform inference; only core sectors enter the reported aggregate. The supplied core weights exclude three sector rows: food and beverages purchased for off-premises consumption; gasoline and other energy goods; and gas/electric utilities. Food services and accommodation remain included under this definition. For nominal spending V_{it} and included set $\mathcal{C}$,

$$w_{it} = \begin{cases} V_{it} / \sum_{j \in \mathcal{C}} V_{jt}, & i \in \mathcal{C}, \\ 0, & i \notin \mathcal{C}. \end{cases} \quad \sum_{i=1}^{17} w_{it} = 1.$$

These weights aggregate inferred trends. They are not factor loadings and do not impose the factor-extraction weights. The model is estimated from sector inflation, not from a weighted average supplied as the sole input.

Do not standardize inputs casually. Replacing each series by a z-score changes the units, priors and aggregation unless the full model and transformation back are adjusted consistently. The supplied estimator receives inflation in percentage units.

Local data limitation. The newer input reconstructs housing excluding gas/electric utilities from housing and water/sanitation using a bilateral Fisher chain. The upstream repository omits its original input-preparation script. That reconstruction is explicitly separate from the exact October 2023 input snapshot. Data revisions also mean a current-vintage fit should not be expected to equal the older published series.

7 What is Bayesian about it?

Let X contain latent trend and transitory states, A the loadings, H log volatilities, R outlier indicators/scales, and Θ static parameters. The target is a distribution,

$$p(X, A, H, R, \Theta \mid Y) \propto p(Y \mid X, A, H, R, \Theta) p(X, A, H, R \mid \Theta) p(\Theta).$$

The likelihood rewards explanations that fit the observations. Transition distributions reward plausible evolution through time. Priors constrain quantities that the data may not identify sharply. The result expresses uncertainty about the entire historical decomposition, not only about a few regression coefficients.

The following are actual hyperparameters in the published driver. We define $\text{IG}(\alpha, \beta)$ by density proportional to $v^{-\alpha-1} \exp(-\beta/v)$ for $v > 0$.

Quantity	Prior specification
Loading innovation variance λ^2	$\text{IG}(\nu_\lambda/2, \nu_\lambda s_\lambda^2/2)$; $\nu_\lambda = 12$, $s_\lambda^2 = 0.25^2/(60 \cdot 12)$.
Log-volatility innovation variance γ^2	Same form; $\nu_\gamma = 60$, $s_\gamma^2 = 1/(60 \cdot 12)$.
Ordinary-shock probability p_j	Beta(117.5, 2.5), with mean 47/48.
MA coefficient proposal	Gaussian regression update with prior precision diag(0.1, 0.4, 0.9), followed by the published root transformation.

The s^2 entries are prior scale parameters, not the prior means of the variances. Initial-state distributions also matter: the initial common trend and common transitory state are fixed at zero; sector trend initial variances are 11 and sector MA-state initial variances are 1. Initial loading covariance matrices are given in the appendix.

For intuition, a small loading-innovation variance favours smoother loading paths. The prior mean for p_j favours rare outliers, without requiring their frequency to remain fixed at 1/48. The observations update these assumptions.

Key idea. Priors do more than supply starting values. Starting values launch the algorithm; priors remain part of the posterior throughout estimation.

8 One sampling sweep, step by step

The published estimator uses a blocked Gibbs-style MCMC procedure. Begin with initial values and an initial draw of the latent states. Then repeat the following sequence; each block conditions on the most recently available values of the others.

1. **Draw the loading paths** a_{it}, b_{it} . Hold common states and sector trends fixed. The remaining observation equation is linear in the loadings. Their random-walk law supplies the transition equation. The code augments this block with sector MA-noise states and uses a simulation smoother, rather than fitting independent regressions at each date.
2. **Draw loading smoothness parameters** λ . Use the increments of the newly drawn loading paths and the inverse-gamma update described below.
3. **Draw trend and transitory states**. Hold loadings, variances, outlier scales and MA coefficients fixed. Equation (2) is now linear Gaussian in the state vector. A simulation smoother draws the whole history jointly, including common and sector-specific trends.
4. **Draw MA coefficients** θ_i . Conditional on the sector innovations, this is a Gaussian regression calculation followed by the source code's root transformation.
5. **Draw the 36 volatility paths**. Use trend increments and transitory innovations (with their current outlier scales removed). A normal-mixture approximation permits Gaussian state smoothing for log variance.
6. **Draw volatility smoothness parameters** γ . Use increments of the newly drawn log-variance paths.
7. **Draw outlier scales** r_{jt} . Compare how well each allowed scale explains the innovation, combining its likelihood with its current prior probability.
8. **Draw ordinary-shock probabilities** p_j . A beta update uses the sampled counts of ordinary and outlier observations.

One sweep produces a revised, mutually conditioned explanation of the data. Repeating sweeps explores alternatives. These draws are not independent bootstrap datasets, and each iteration is not a new month of observed data. The entire observed sample remains fixed during a fit.

The source uses $t_{\text{skip}} = 4$ for several parameter updates and treats the first four innovations as missing in volatility/outlier updates. It still uses those months' inflation observations in the state smoother. This is different from deleting four rows from the model.

9 What the Kalman filter and simulation smoother actually do

For fixed values of the conditioning parameters, write

$$\mathbf{y}_t = Z_t \mathbf{x}_t + \boldsymbol{\epsilon}_t, \quad \boldsymbol{\epsilon}_t \sim \mathcal{N}(0, H_t), \quad \mathbf{x}_t = F_{t-1} \mathbf{x}_{t-1} + G_{t-1} \boldsymbol{\eta}_{t-1}, \quad \boldsymbol{\eta}_{t-1} \sim \mathcal{N}(0, Q_{t-1}).$$

Given the previous filtered mean m_{t-1} and covariance C_{t-1} , the forward filter computes

$$\begin{aligned} a_t &= F_{t-1} m_{t-1}, & P_t &= F_{t-1} C_{t-1} F_{t-1}' + G_{t-1} Q_{t-1} G_{t-1}', \\ v_t &= \mathbf{y}_t - Z_t a_t, & S_t &= Z_t P_t Z_t' + H_t, \\ K_t &= P_t Z_t' S_t^{-1}, & m_t &= a_t + K_t v_t, \\ & & C_t &= P_t - K_t S_t K_t'. \end{aligned}$$

This is a sequence of Gaussian conditioning calculations. The Kalman gain determines how much the new observation changes the prior state prediction, given its uncertainty and the observation's noise.

Filtering versus smoothing. Filtering gives $p(\mathbf{x}_t | Y_{1:t}, \Theta)$. Smoothing uses the full sample and gives $p(\mathbf{x}_t | Y_{1:T}, \Theta)$. A later observation can therefore change the historical estimate for an earlier month.

Smoothing a mean versus drawing a path. The Gibbs algorithm needs a draw from the joint conditional distribution $p(\mathbf{x}_{0:T} | Y, \Theta)$, not just its mean. Replacing every draw by a mean would discard state uncertainty and would no longer be the same sampler. Nor can one independently draw from each date's marginal distribution: the cross-time covariance matters.

One Gaussian simulation-smoothing construction is:

1. Generate auxiliary states and observations (X^+, Y^+) under the current model.
2. Smooth the residual observations $Y - Y^+$ under the corresponding zero-mean formulation.
3. Add the smoothed correction to X^+ .

The resulting path has the required conditional distribution when initial means and covariances are handled consistently. This is the principle behind the upstream simulation smoother. Our Python implementation delegates the general filtering and simulation-smoothing calculations to `statsmodels`; it assembles the MCT-specific matrices and updates.

10 The other Bayesian updates, in equations

Innovation variances. Suppose $d_t \mid v \sim \mathcal{N}(0, v)$ and $v \sim \text{IG}(\nu/2, \nu s^2/2)$. With m included increments,

$$v \mid d \sim \text{IG}\left(\frac{\nu + m}{2}, \frac{\nu s^2 + \sum_t d_t^2}{2}\right).$$

This is used for loading-increment variances and log-volatility-increment variances. The code returns a standard deviation, $\sqrt{v}$, after drawing the variance through a reciprocal gamma calculation.

MA coefficients. For the rescaled conditional regression $\tilde{\mathbf{u}} = X\boldsymbol{\theta} + \boldsymbol{\varepsilon}$ with unit error variance and zero-mean prior precision P ,

$$V_\theta = (P + X'X)^{-1}, \quad m_\theta = V_\theta X' \tilde{\mathbf{u}}, \quad \boldsymbol{\theta}^* \sim \mathcal{N}(m_\theta, V_\theta).$$

The upstream code uses a pseudoinverse and then transforms polynomial roots to enforce its MA convention. Thus the final update is not merely an unconstrained normal draw; the port reproduces that particular transformation. It should not be described as generic rejection sampling from a truncated Gaussian posterior.

Outlier allocations. For standardized innovation $x_t = e_t/\sigma_t$ and candidate multiplier r_k ,

$$\Pr(r_t = r_k \mid x_t, \dots) \propto \Pr(r_t = r_k) \frac{1}{r_k} \exp\left(-\frac{x_t^2}{2r_k^2}\right).$$

If n_1 sampled scales equal 1 and n_o exceed 1, then

$$p \mid R \sim \text{Beta}(117.5 + n_1, 2.5 + n_o).$$

Stochastic volatility. For $x_t = \sigma_t z_t$, taking log squares gives approximately

$$\log(x_t^2 + 10^{-3}) = h_t + \log z_t^2.$$

The $\log\text{-}\chi_1^2$ error is non-Gaussian. The code approximates it using ten normal mixture components with fixed probabilities, means and variances (following Omori et al.). Conditional on sampled mixture labels, subtract the selected means and use their variances as observation noise. The log-variance random walk then becomes a Gaussian state-space problem. This mixture approximation and the small logarithm offset are part of the implemented algorithm; the procedure is not an exact closed-form solution to the full nonlinear model.

11 Turn posterior draws into the reported inflation measure

For retained draw s , compute sector trends and their aggregate:

$$g_{it}^{(s)} = a_{it}^{(s)} \tau_{ct}^{(s)} + \tau_{it}^{(s)}, \quad M_t^{(s)} = \sum_i w_{it} g_{it}^{(s)}.$$

The reported estimate is the median of $\{M_t^{(s)}\}_s$. The supplied code uses quantiles 1/6 and 5/6 for an interval containing 2/3 of posterior mass; website descriptions round the band to approximately 68%. The posterior includes uncertainty about states and parameters, conditional on the chosen model and data vintage. It does not automatically include uncertainty over all possible alternative models.

A one-month, one-draw example. Suppose there are three included sectors with weights (0.5, 0.3, 0.2). Let $\tau_c = 2$, loadings (1, 0.8, 1.2) and sector trends (0.4, -0.1, 0.2). Then

$$\mathbf{g} = (2.4, 1.5, 2.6)', \quad M = 0.5(2.4) + 0.3(1.5) + 0.2(2.6) = 2.17\%.$$

The common contribution is $2[0.5(1) + 0.3(0.8) + 0.2(1.2)] = 1.96$ percentage points; the specific contribution is $0.5(0.4) + 0.3(-0.1) + 0.2(0.2) = 0.21$. Temporary innovations do not enter this aggregate.

Repeat this calculation for every draw. **Aggregate first, then take posterior quantiles.** In general,

$$\text{median}\left(\sum_i w_i g_i\right) \neq \sum_i w_i \text{median}(g_i).$$

This is why displayed sector contribution medians need not add exactly to the aggregate median. Draw-level contributions do add exactly.

Normalization. The original driver rescales the common trend and its loadings after sampling: $a_{it}^* = a_{it} \sigma_{\Delta\tau, c, t}$ and $\tau_{ct}^* = \tau_{ct} / \sigma_{\Delta\tau, c, t}$. Their product is unchanged, so the Python aggregate can use the unnormalized products. This is a reporting transformation; one should not assume the transformed factor retains the original random-walk transition equation unchanged.

Trend versus forecast. MCT is an inferred persistent component. It is not automatically the forecast of next month's headline inflation. Forecasting requires propagating posterior states and parameter uncertainty forward, adding the transitory dynamics, and specifying future aggregation weights as appropriate.

12 Convergence, validation, and computational cost

The published driver sets 3,000 burn-in outer iterations, 3,000 retained draws, and two inner sweeps per outer iteration. Its loop also includes iteration zero, so it executes 12,002 sweeps. Burn-in discards the initial portion of the chain to reduce dependence on the starting values; a fixed burn-in length is not proof that this dependence has disappeared. Thinning stores fewer, more widely spaced draws but does not make them independent or guarantee efficient computation.

Each sweep repeatedly processes all months, including two large multivariate state problems and 36 scalar volatility paths. The main state has 87 coordinates and the loading block 102 for the 17-sector MA(3) specification. Thousands of full-history simulation-smoothing calculations explain the hours-long run, despite individual sweeps being fairly quick. The Python implementation is much faster than the initial Octave execution, but it is not claimed to be the fastest possible implementation.

What should count as validation?

1. Check input definitions, seasonal adjustment, date alignment and expenditure weights. Verify that every month's included weights sum to one.
2. Check state-space matrices and conditional calculations against the source. Our selected initialization/matrix and smoother fixtures pass; this checks components of the implementation.
3. Compare the full October 2023 fit with the NY Fed output using the same data vintage. Numerical differences across seeds and software are expected; quantify them rather than asserting equality.
4. Run multiple chains with different starting points/seeds. Examine trace plots, rank-normalized $\hat{R}$, effective sample sizes and Monte Carlo standard errors for the aggregate and other identified quantities. Raw factor signs/scales can obscure diagnostics.
5. Assess sensitivity to priors and influential observations, and inspect posterior predictive behaviour. A narrow interval from a poorly mixed chain is not useful evidence of precision.

The current compact exporter stores aggregate draws and summaries, not every latent parameter draw. More extensive parameter diagnostics require expanding the saved output. Multiple-chain convergence and full replication accuracy remain to be established; the document makes no claim that the running fit has already passed them.

Evaluation differs from VAR forecasting. AIC is not the main validation tool for this latent Bayesian decomposition. If your goal is forecasting, evaluate genuine out-of-sample forecasts using rolling origins, preferably with historical data vintages. Do not use a full-sample smoothed historical MCT as if it had been available in real time: it incorporates later observations and revisions.

A The state vector and an implementation map

For three MA lags, the main state at t can be ordered as

$$\mathbf{x}_t = (\tau_{ct}, \tau_{1t}, \dots, \tau_{nt}, e_{ct}, e_{1t}, e_{1,t-1}, e_{1,t-2}, e_{1,t-3}, \dots, e_{nt}, \dots, e_{n,t-3})'.$$

Its dimension is $1 + n + 1 + 4n = 5n + 2 = 87$. The observation row for sector i picks out its common trend with coefficient a_{it} , its own trend with coefficient 1, common noise with coefficient b_{it} , and its four noise states with coefficients $(1, \theta_{i1}, \theta_{i2}, \theta_{i3})$.

The transition matrix retains each trend, resets the new transitory innovations, and shifts each sector's three noise lags down by one position. A selection matrix injects $2(n + 1) = 36$ independent shocks into the common/sector trends and current transitory innovations. The lagged noise coordinates do not get fresh independent shocks.

The loading block includes n trend loadings, n transitory loadings and $4n$ noise coordinates, for $6n = 102$ states. Its observation data are sector inflation minus the currently drawn sector trends. Its time-varying design uses the current common trend and common transitory shock as regressors. The initial loading covariance is $11I_n$ for trend loadings and $I_n + 10\mathbf{1}\mathbf{1}'$ for transitory loadings; MA-noise initial covariance is I_{4n} .

Local file or object	Role
<code>prepare_inputs.py</code>	Inflation transformations, dates, sector mapping and normalized weights.
<code>mct_python.py</code> : <code>MCT</code>	Model-specific state matrices and Gibbs update schedule.
<code>GaussianStates</code>	Thin adapter to statsmodels simulation smoothing.
<code>VolatilitySampler</code>	Mixture allocation and package-based log-volatility smoothing.
<code>run_python.py</code>	Settings, execution, progress and saved draws.
<code>export_results.py</code>	Posterior summaries, reference comparisons and workbooks.
<code>validate_python.py</code>	Source-based matrix/smoothing checks.
<code>upstream/</code>	Unmodified NY Fed reference source and snapshot.

In the original code, a, b are `alpha_tau, alpha_eps`; τ_c, τ_i are `tau_c, tau_i`; e_c, e_i are `eps_c, eps_i`; and r is `s_eps`. The original SSM.H is the observation design matrix; it is called `design` in statsmodels and Z_t in this guide. Letter conventions differ between libraries.

B Reading route and sources

Start with the NY Fed FAQ for the target of the model. Read the introduction in Section 1 of this guide before the technical papers if factors still feel unfamiliar. Use the published code to resolve exact implementation details: the earlier Stock–Watson paper is foundational, but is not a complete specification of every later NY Fed modification.

The code used as reference in this project is pinned to commit:

```
7e68e46e2ff4ad55b712f2c6c49f08f9c48e1ea1
```

Local copies of the research material are in `../references/` relative to this document’s directory.

References

- [1] Federal Reserve Bank of New York. *Multivariate Core Trend Inflation: overview and FAQ*. [Open model description](#).
- [2] Federal Reserve Bank of New York, Time Series Analysis Team. *MCT-PCE replication code*. [Open repository](#). In particular: `automated_PCE.m`, `estimate_MCT.m`, `update_vol.m`, `update_gam.m`, `update_theta.m`, `update_scl.m`, and `update_ps.m`.
- [3] Almuzara, M., and A. M. Sbordone (2024). *Measurement and Theory of Core Inflation*. NY Fed Staff Report 1115. [Open PDF](#).
- [4] Stock, J. H., and M. W. Watson (2016). *Core Inflation and Trend Inflation*. *Review of Economics and Statistics*, 98(4), 770–784. [Open paper](#); [open technical appendix](#).
- [5] Almuzara, M., and A. M. Sbordone (2022). *Inflation Persistence: How Much Is There and Where Is It Coming From?* Liberty Street Economics. [Open introduction](#).
- [6] Statsmodels. *SimulationSmoother documentation*. [Open package documentation](#). Python implementation uses version 0.14.6.

Key idea. To understand one iteration, temporarily treat every unknown except one block as known. That block becomes a familiar regression, Gaussian state-space calculation, or conjugate probability update. MCMC connects those conditional calculations into joint inference.